

July 17th, 2026

Hospital Fixed-Asset Tracking System – System Requirement Document

Imperial Healthtech

Document Version 5.0

Supervisors: *Kemal Falatehan*
Choirunnisa Hapsari

Author: Farros Ramzy

Abstract

This document specifies the system-level requirements for the Hospital Fixed-Asset Tracking System. The product uses UHF RFID tags, ESP32-based sensor nodes, MQTT communication, a backend processing platform, persistent data storage, and browser-based registration and monitoring applications. The specification defines the functional behavior, interfaces, data rules, quality attributes, safety constraints, deployment requirements, and acceptance criteria required to support hospital asset registration, location monitoring, controlled movement, alerts, and node management.

Version Table

Version	Date of Creation/Change	Changes	Reviewer(s)	Approval Status
0.0	30/03/2026	Document initiation		
0.5	01/04/2026			
1.0	02/04/2026			
1.1	06/04/2026			
1.2	13/04/2026			
1.5	29/05/2026			

Table of Contents

Abstract	I
Table of Contents.....	i
List of Figures.....	iii
List of Tables	iv
1. Introduction.....	1
1.1. Purpose.....	1
1.2. Scope and System Boundary.....	1
1.3. Definitions and Acronyms.....	1
1.4. References	2
2. Overall Description	2
2.1. Product Perspective	2
2.2. Major Product Components	3
2.3. User Classes & Roles	4
2.4. Operating Environment	4
2.5. Assumptions and Dependencies.....	4
2.6. Constraints.....	5
3. Software Architecture Overview.....	5
4. User Requirements	6
5. Functional Requirements.....	7
5.1. Embedded Firmware	9
5.2. Backend System	9
5.3. Registration App	9
5.4. Monitor App	10
5.5. User and Access Management.....	10
5.6. System Settings.....	10
5.7. Cross-System Workflows.....	11
6. External Interface Requirements	12
6.1. Hardware Interfaces	12
6.2. Software and API Interfaces.....	13
6.3. Communication Interfaces.....	13
6.4. User Interface Requirements	15
7. Data Requirements	15
7.1. Main Data Entities	15
7.2. Data Validation and Integrity	17
7.3. Time and Date Handling	17
7.4. Retention, Backup, and Recovery	17
8. Non-Functional Requirements.....	18

8.1. Security and Privacy.....	19
8.2. Reliability and Availability	19
8.3. Performance	19
8.4. Usability and Accessibility.....	20
8.5. Scalability.....	20
8.6. Maintainability and Testability.....	20
8.7. Compatibility and Portability	20
8.8. Data Integrity.....	21
8.9. Auditability	21
9. Safety, Privacy, and Compliance Requirements.....	21
9.1. List of Safety Requirements	21
9.2. List of Privacy Requirements.....	22
9.3. List of Compliance Requirements	22
10. Error Handling and Recovery Requirements	22
11. Deployment and Configuration Requirements	23
12. Logging, Audit, and Operational Monitoring	26
13. Verification & Acceptance Criteria	27
14. Requirements Traceability Matrix.....	30
Appendices	32
Appendix A – Status and Role Enumerations.....	32
Appendix B – Recommended Figures.....	32
Appendix C – Open Decisions and TBD Values	32

List of Figures

Figure 2. 1. 1 System context diagram.	3
Figure 3. 1 High-Level Software Architecture.....	6
Figure 6. 1. 1 Sensor Node Hardware Wiring Diagram.....	12
Figure 6. 3. 1 System Communication Flow Diagram.	14
Figure 7. 1. 1 Entity-Relationship Diagram.	16
Figure 11. 1 Complete System Deployment Diagram.....	24
Figure 11. 2 Simplified Complete System Deployment Diagram.	26

List of Tables

Table 1. 2. 1 Project in-scope & out-of-scope list table.	1
Table 1. 3. 1 List of definitions.....	1
Table 1. 4. 1 List of document references.....	2
Table 2. 2. 1 List of product components and their responsibilities.	3
Table 2. 3. 1 List of user roles and their responsibilities.....	4
Table 2. 4. 1 List of subsystem operating environments.....	4
Table 2. 5. 1 Product assumptions list.	4
Table 2. 6. 1 List of constraints.	5
Table 3. 1 List of system interactions.....	6
Table 4. 1 List of user requirements.	6
Table 5. 1 List of functional requirements.....	7
Table 5. 1. 1 List of functional requirements for embedded firmware.	9
Table 5. 2. 1 List of functional requirements for backend system.	9
Table 5. 3. 1 List of functional requirements for registration app.	9
Table 5. 4. 1 List of functional requirements for monitor app.....	10
Table 5. 5. 1 List of functional requirements for user and access management.....	10
Table 5. 6. 1 List of functional requirements for system settings.	10

Table 5. 7. 1 List of expected results from the integrated system workflow.....	11
Table 6. 1. 1 List of hardware interface requirements.....	12
Table 6. 2. 1 List of software interface requirements.	13
Table 6. 3. 1 List of system communication interface requirements.	13
Table 6. 4. 1 List of user interface requirements.	15
Table 7. 1. 1 List of data requirements for system entities and their important data.....	15
Table 7. 2. 1 List of data requirements for system data validation.	17
Table 7. 3. 1 List of data requirements for system date and time formatting.....	17
Table 7. 4. 1 List of data requirements for system recovery.	17
Table 8. 1 List of non-functional system requirements.	18
Table 8. 1. 1 List of non-functional requirements for security and privacy.....	19
Table 8. 2. 1 List of non-functional requirements for system reliability.....	19
Table 8. 3. 1 List of non-functional requirements for system performance.....	19
Table 8. 4. 1 List of non-functional requirements for system usability.	20
Table 8. 5. 1 Non-functional requirements for system scalability.....	20
Table 8. 6. 1 List of non-functional requirements for system maintainability.....	20
Table 8. 7. 1 List of non-functional requirements for system compatibility.....	20
Table 8. 8. 1 List of non-functional requirements for data integrity.	21

Table 8. 9. 1 Non-functional requirement for system auditability.	21
Table 9. 1 List of safety requirements.	21
Table 9. 2 List of privacy requirements.....	22
Table 9. 3 List of compliance requirements.....	22
Table 10. 1 List of system error handling and recovery requirements.....	22
Table 11. 1 List of system deployment and configuration requirements.....	23
Table 11. 2 List of relations and communication lines of the system deployment diagram.	25
Table 12. 1 System data log requirements.	26
Table 13. 1 List of system verification and acceptance criteria.	27
Table 14. 1 Matrix table of user requirements with use case and system acceptance evidence.....	30
Table 15. 1 System status and role rules.	32
Table 15. 2 Document figures.	32
Table 15. 3 List of decision plans for the product.....	32

1. Introduction

This document provides the authoritative system-level requirements baseline for the development, verification, deployment, and future maintenance of the Hospital Fixed-Asset Tracking System Project's product. This section introduces the purpose, scope, terminology, and reference documents used by this specification.

1.1. Purpose

The purpose of this document is to specify the requirements for the complete Hospital Fixed-Asset Tracking System. It defines what the Embedded Firmware, Backend System, Registration App, Monitor App, database, and communication services shall provide, along with the constraints and quality attributes to keep their operation in scope.

This specification is intended for project supervisors, software and embedded engineers, testers, deployment personnel, and future maintainers. Detailed actor interactions and scenario flows are maintained in the companion Requirements and Use Case Specification. In contrast, detailed sequence, state-machine, class, and deployment diagrams belong in the Software Design Specification.

1.2. Scope and System Boundary

The product tracks RFID-tagged movable hospital assets by combining registration-desk scanning, room-based checkpoint detections, backend processing, and browser-based monitoring. The system boundary includes the sensor-node firmware, MQTT messaging, backend API and services, PostgreSQL database, Registration App, Monitor App, user and access management, alerts, movement workflows, and configurable system settings.

The system is intended to support asset visibility and operational management. It does not diagnose patients, monitor patient vital signs, make treatment decisions, replace clinical procedures, or control medical equipment. Patient clinical data and medicine requirements are outside the intended scope.

The Scope boundaries can be understood as follows:

Table 1. 2. 1 Project in-scope & out-of-scope list table.

No.	In scope	Out of Scope
1.	RFID tag registration and deregistration.	Clinical diagnosis or treatment decisions.
2.	Checkpoint-node discovery, assignment, status, and heart-beat monitoring.	Procurement, billing, or inventory purchasing.
3.	Asset location, expected room, movement request, and wrong-location warnings.	Patient and Medical Staff location, or their identity tracking.
4.	Role-based user access, system settings, alerts, and activity records.	Automatic control of medical equipment.
5.	MQTT node communication, backend processing, and Web-Socket updates	Guaranteeing RFID coverage in every physical environment without site testing.

1.3. Definitions and Acronyms

In this document, the following terms are used to refer to certain parts of the system. Below is a list of terms and their definitions.

Table 1. 3. 1 List of definitions.

Term/Acronym	Definition
Asset	A movable hospital item registered in the system and associated with a unique RFID tag ID.
UHF RFID	An Ultra-High-Frequency Radio-Frequency Identification technology for reading tagged assets.
Registration Node	An ESP32-based node for scanning RFID tags during asset registration and deregistration.

Checkpoint Node	An ESP32-based node deployed in a hospital room or at a checkpoint to detect tagged assets and report heartbeat messages.
Node	A Registration Desk Node or a Checkpoint Node recognized by the backend.
Assigned Room	The room in which the asset is expected to be located.
Current Room	The room associated with the most recent valid checkpoint detection for the asset.
Pending Placement	An asset state is used after registration until the asset is detected in its assigned room.
Movement Request	A request to move an asset from its current or assigned location to a destination room.
Unauthorized Movement	A detection outside the expected location without a valid approved movement workflow.
Heartbeat	A periodic node message indicating that the node is operational and connected.
MQTT	A publish/subscribe messaging protocol used between sensor nodes and the backend.
WebSocket	A persistent connection used by the backend to deliver live updates to authenticated frontend clients.
API	Application Programming Interface used by the web applications to communicate with the backend.
JWT	JSON Web Token is used to represent an authenticated user session.
WIB	Western Indonesian Time, UTC+7, is used for user-facing time display where configured.

1.4. References

The following project documents and technical sources provide supporting requirements, design details, hardware constraints, or verification evidence. Detailed use-case flows and design diagrams are intentionally maintained outside this specification.

Table 1. 4. 1 List of document references.

Reference ID	Document / Source	Purpose
REF-001	Hospital Fixed-Asset Tracking System – Project Plan Document	Defines the system plan and elaborates on the initial architecture, scope, and design sketch.
REF-002	Hospital Fixed-Asset Tracking System – Requirements and Use Case Specification	Defines user requirements, actors, use-case diagrams, detailed flows, and system functional and non-functional requirements, as well as mappings of use-case relations.
REF-003	Hospital Fixed-Asset Tracking System – System Design Document	Defines the architecture details, sequence diagrams, state machines, class diagrams, communication protocols, database, and deployment design.
REF-004	Hospital Fixed-Asset Tracking System – System Product Report Document	Provides the system production report, together with the list of tools, how the research is going, where the user and system requirements are needed, and the list of test results of the product.
REF-005	EL-UHF-RMT01 – Protocol Documentation (written in Indonesia)	Provides the communication protocol, PIN address, and command ID of the UHF-RMT01-RFID sensor hardware.
REF-006	EL-UHF-RMT01 – Datasheet (written in Indonesia)	Provides the operating constraints, pins, and wiring of the UHF-RMT01-RFID sensor hardware.
REF-007	ESP32-WROOM-32 - Datasheet	Provides the operating constraints, pins, communication protocols, and electrical connections of the ESP32 microcontroller.

2. Overall Description

This section provides the overall product context, the principal components and users, and the environment records, assumptions, dependencies, and constraints that affect implementation and deployment.

2.1. Product Perspective

The Hospital Fixed-Asset Tracking System is a distributed client-server and embedded system. Physical sensor nodes read RFID tags and publish node events through MQTT. A FastAPI backend validates the messages, applies asset and movement rules, updates PostgreSQL records, exposes authenticated APIs, and broadcasts live events through WebSocket. The Registration App supports administrative and registration-side workflows, while the Monitor App provides operational visibility and movement-request functions.

The browser applications do not communicate directly with the MQTT broker. MQTT access is limited to the backend and sensor nodes. This preserves a backend-centered security and business-rule boundary and avoids exposing broker credentials or raw node topics to end users.

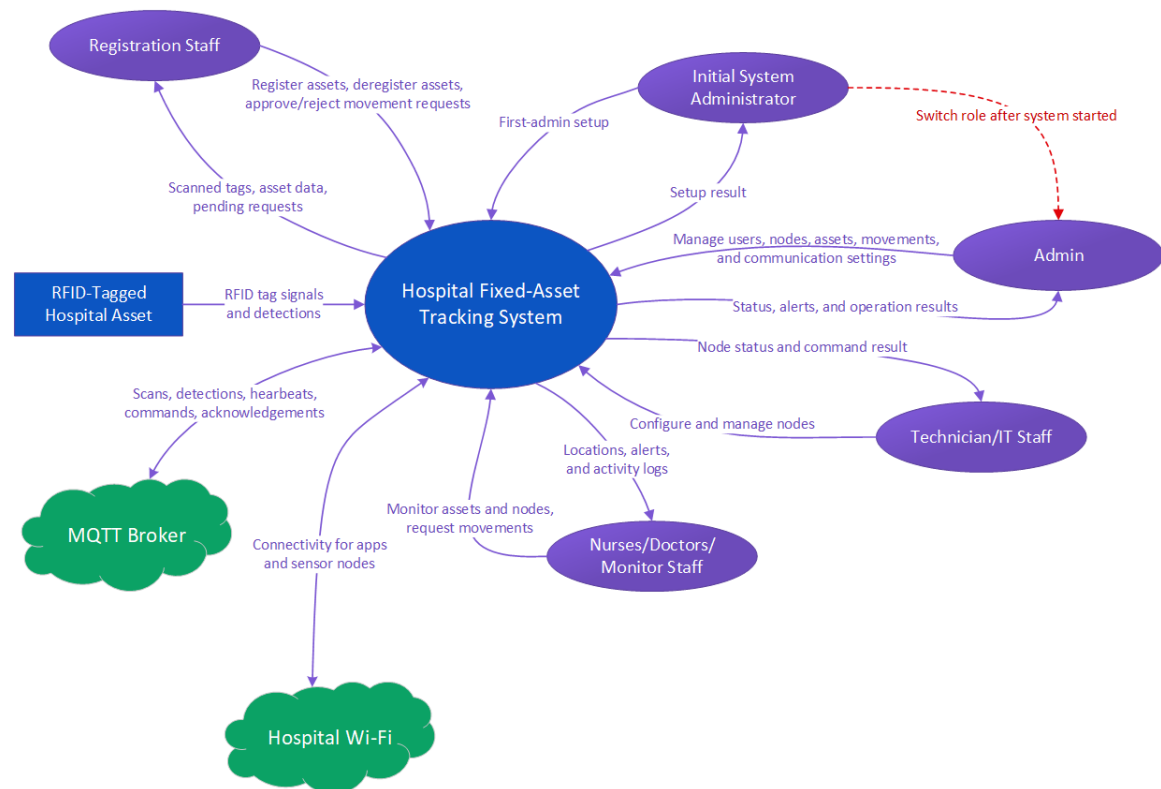


Figure 2. 1. 1 System context diagram.

2.2. Major Product Components

The following table identifies the principal product components, their external interfaces, and their primary responsibilities. Internal class and module structures are defined in the System Design Document.

Table 2. 2. 1 List of product components and their responsibilities.

No.	Component	Main Interfaces	Primary Responsibility
1.	Embedded Firmware	- RFID reader - Wi-Fi - MQTT - Status LEDs - Local Setup Page	Wi-Fi setup, node provisioning, RFID scanning or detection, heartbeat publication, command reception, LED indication, and MQTT reconnection.
2.	Backend System	- HTTPS API - WebSocket - MQTT - PostgreSQL	Authentication, authorization, node and asset services, movement workflows, alert generation, MQTT processing, WebSocket broadcasting, and persistence.
3.	Registration App	- HTTPS API - WebSocket	First-admin setup, login, role-based dashboard, node management, asset registration and deregistration, movement decisions, user management, password change, and settings.
4.	Monitor App	- HTTPS API - WebSocket	Dashboard, asset and node monitoring, alerts, activity logs, movement requests, and live updates.
5.	PostgreSQL Database	SQLAlchemy/database connection	Persistent storage for users, nodes, assets, detections, movement requests, alerts, activity records, and settings.
6.	MQTT Broker	MQTT publish/subscribe	Message delivery between nodes and the backend.

2.3. User Classes & Roles

Human users receive role-based access through the web applications. The Initial System Administrator is a temporary setup actor used only when no admin account exists; after setup, the created account operates as a normal Admin account.

Table 2. 3. 1 List of user roles and their responsibilities.

No.	Role	Typical Access	Responsibilities
1.	Initial System Administrator	- First-Admin and App Setup - All authorized product functions	Creates the first admin account when no admin exists.
2.	Administrator	All authorized product functions	Manages users, nodes, assets, movement decisions, dashboards, and system settings.
3.	Test User	- Node Management Page - Asset Management Page - Monitor App - Change Password Page - App Communication Setting	Exercises most operational functions for testing and demonstration, without full user account administration.
4.	Technician	- Registration App - Node Management Page - Change Password Page	Installs, discovers, assigns, identifies, edits, unassigns, and removes sensor nodes.
5.	Registration Staff	- Registration App - Asset Management Page - Change Password Page	Registers and deregisters assets and approves or rejects movement requests.
6.	Monitor Staff	- Monitor App - Change Password Page of the Registration App	Observes assets, nodes, alerts, and activity, and requests or cancels movements.

2.4. Operating Environment

Each subsystem runs in a defined hardware or software environment and depends on network access to the interfaces it uses. The following environments are the intended baseline for development and deployment.

Table 2. 4. 1 List of subsystem operating environments.

No.	Subsystem	Operating Environment
1.	Sensor Nodes	ESP32-compatible controller, selected UHF RFID reader, stable power supply, Wi-Fi coverage, and PlatformIO-supported firmware environment.
2.	Backend	Python-based FastAPI runtime, PostgreSQL database, MQTT client library, environment variable configuration, and HTTPS-capable local or cloud hosting.
3.	Registration App	Modern Browser app on a desktop, tablet, or a supported mobile display; network access to the backend API and WebSocket endpoint.
4.	Monitor App	Modern browser on a desktop, tablet, or a supported mobile display; network access to the backend API and WebSocket endpoint.
5.	Database	PostgreSQL instance with managed credentials, migrations, backup capability, and backend network access.
6.	MQTT Broker	Broker reachable from the backend and sensor nodes, with configured topics, credentials, and connection parameters.

2.5. Assumptions and Dependencies

Assumptions and dependencies identify conditions expected to be true for normal system operation. They are not substitutes for requirements and shall be validated during installation or deployment, where applicable.

Table 2. 5. 1 Product assumptions list.

No.	ID	Assumption / Dependency
1.	A-001	Each tracked asset has one compatible and readable UHF RFID tag.
2.	A-002	Wi-Fi coverage is available where Registration Desk Nodes and Checkpoint Nodes are installed.
3.	A-003	The MQTT broker is reachable by both sensor nodes and the backend.
4.	A-004	The backend can connect to the PostgreSQL database and run the required migrations.
5.	A-005	Hospital rooms, hospitals, and node assignments are configured before normal tracking.
6.	A-006	Users have unique active accounts and assigned roles before accessing protected pages.

7.	A-007	RFID antenna placement and tag orientation are validated during site installation.
8.	A-008	System clocks and time zones are configured consistently enough to compare heartbeat and event timestamps.
9.	A-009	Browser clients support modern JavaScript, HTTPS, and WebSocket connections.
10.	A-010	Cloud-service limitations, including free-tier suspension or sleep, are managed operationally or replaced in production.

2.6. Constraints

Constraints limit the available implementation and deployment choices. They include selected hardware, network dependence, browser-to-backend boundaries, environmental RFID limitations, safety obligations, secret handling, authorization, modularity, and hosting limitations.

Table 2. 6. 1 List of constraints.

No.	ID	Constraint
1.	C-001	Firmware shall run on the selected ESP32-compatible hardware and remain compatible with the chosen RFID reader.
2.	C-002	Sensor nodes depend on Wi-Fi and MQTT connectivity; loss of connectivity may delay event delivery.
3.	C-003	Browser applications shall use the backend API and WebSocket interfaces rather than connecting directly to MQTT.
4.	C-004	RFID performance is affected by metal, liquids, antenna placement, tag orientation, and radio interference.
5.	C-005	The system shall not interfere with clinical work or create electrical, trip, or installation hazards.
6.	C-006	Production secrets shall not be stored in public source code or distributed firmware.
7.	C-007	The application shall preserve role-based authorization in both frontend navigation and backend endpoints.
8.	C-008	The implementation should remain modular, with small focused files and reusable services, hooks, and utilities.
9.	C-009	Free-tier or development hosting may not satisfy production availability requirements.

3. Software Architecture Overview

The system uses a distributed architecture with embedded nodes, a backend-centered application layer, persistent storage, and two browser clients. Sensor nodes publish RFID scans, detections, heartbeats, and command acknowledgments through MQTT. The backend subscribes to the required topics, validates incoming payloads, applies business rules, stores changes, and broadcasts supported WebSocket events. The browser applications call authenticated HTTP APIs to execute commands and retrieve data, and listen for WebSocket events to receive live updates.

The image below shows how the system is interconnected between its unit partitions. The table below the image explains the interactions, communication protocol, source, and destination. However, detailed component, class, sequence, state-machine, and deployment diagrams are maintained in the Software Design Specification. This section provides only the architectural context required to understand the requirements.

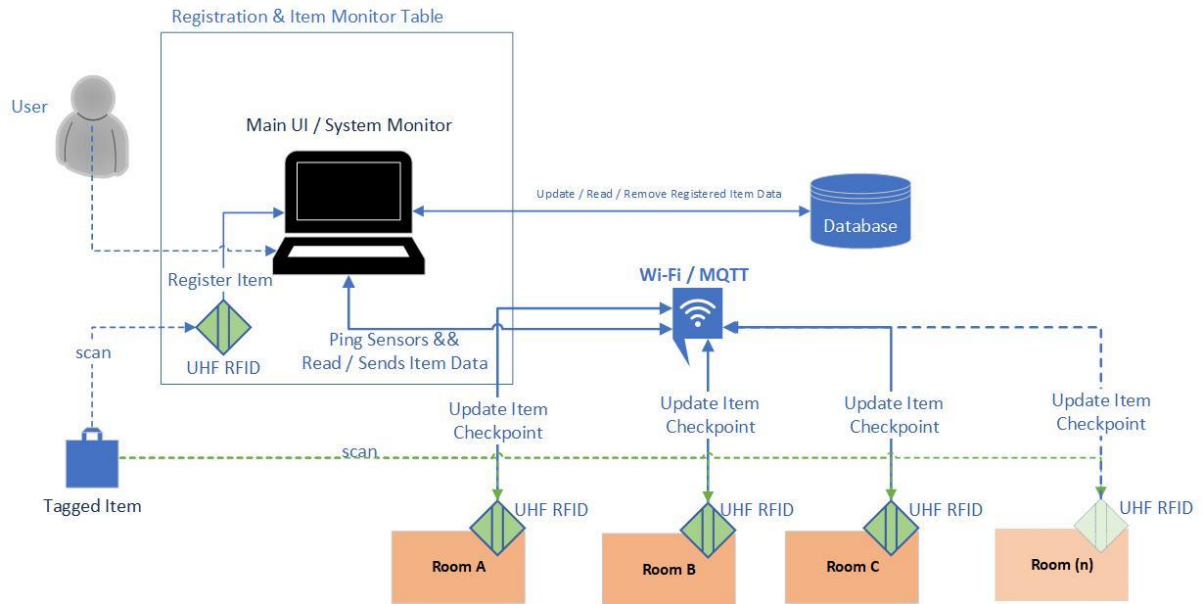


Figure 3. 1 High-Level Software Architecture.

Table 3. 1 List of system interactions.

No.	Interaction	Source	Destination	Protocol / Mechanism
1.	User commands and data requests	Registration App / Monitor App	Backend API	HTTPS with JSON and bearer token
2.	Live system events	Backend	Authenticated browser clients	WebSocket
3.	RFID scans, detections, and heartbeats	Sensor Nodes	Backend through broker	MQTT
4.	Node blink commands	Backend	Target the sensor node through the broker	MQTT
5.	Persistent reads and writes	Backend	PostgreSQL	Database connection through SQLAlchemy

4. User Requirements

User requirements describe the capabilities the product shall provide from the perspective of administrators, technicians, registration staff, monitoring staff, and supporting operational users. They are traced to functional requirements and acceptance evidence in Section 14.

Table 4. 1 List of user requirements.

No.	ID	User Requirement Description
1.	UR-001	The system shall allow authorized users to log in, log out, and access only the pages allowed by their assigned role.
2.	UR-002	The system shall allow all authenticated users to change their own passwords securely.
3.	UR-003	The system shall allow the admin to create, update, disable, and manage user accounts and roles.
4.	UR-004	The system shall display a role-based Registration Dashboard with only the modules and summaries available to the logged-in user.
5.	UR-005	The system shall allow authorized users to open the Node Registration page and manage hospital tracking nodes.
6.	UR-006	The system shall allow authorized users to view records of discovered, assigned, online, offline, disabled, and unknown nodes.
7.	UR-007	The system shall allow authorized users to assign, edit, unassign, delete, and identify physical nodes by blink command.
8.	UR-008	The system shall allow authorized users to open the Asset Registration page and manage registered assets.
9.	UR-009	The system shall allow authorized users to register a new RFID-tagged hospital asset using a registration desk node.
10.	UR-010	The system shall allow authorized users to deregister an existing active asset when it is no longer tracked.
11.	UR-011	The system shall allow monitor-side users to request and cancel asset movement between rooms.

12.	UR-012	The system shall allow registration-side users to approve or reject pending asset movement requests.
13.	UR-013	The system shall complete an approved asset movement when the asset is detected at the approved destination checkpoint node.
14.	UR-014	The system shall allow monitoring users to view asset location, assigned room, movement status, node status, alerts, and activity logs.
15.	UR-015	The system shall detect and show warnings for unknown RFID tags, offline nodes, and unauthorized asset movement.
16.	UR-016	The system shall receive RFID scans, RFID detections, and heartbeat messages from sensor nodes through MQTT.
17.	UR-017	The system shall broadcast live updates to the Registration App and Monitor App without requiring manual refresh.
18.	UR-018	The system shall prevent unauthorized roles from accessing restricted pages or performing restricted actions.
19.	UR-019	The system shall allow authorized users to view and manage system or MQTT configuration settings.
20.	UR-020	The system shall allow an initial system administrator to create the first admin account when no admin account exists.

5. Functional Requirements

Functional requirements define the mandatory behavior of the system. Requirement identifiers are retained to remain consistent with the companion Use Case Specification and its traceability matrices. Each requirement shall be verified using inspection, demonstration, unit testing, integration testing, or end-to-end testing as indicated.

The table below defines the functional requirements of the Hospital Fixed-Asset Tracking System product. The following subsections group the requirements by primary subsystem and verification method.

Table 5. 1 List of functional requirements.

No.	ID	Functional Requirement Description
1.	FR-001	The system shall provide a login function for authorized users in the Registration App and Monitor App.
2.	FR-002	The system shall verify the user account, password, active status, and role during login.
3.	FR-003	The system shall return the authenticated user's token and role after a successful login.
4.	FR-004	The system shall allow logged-in users to sign out and clear their active session data.
5.	FR-005	The system shall allow authenticated users to change their own password by entering the current password, new password, and confirmation password.
6.	FR-006	The system shall verify the current password before updating the stored password hash.
7.	FR-007	The system shall allow the admin to create, update, disable, and manage user accounts.
8.	FR-008	The system shall prevent unsafe admin account actions, such as disabling or demoting the last active admin account.
9.	FR-009	The system shall restrict protected pages and actions based on the authenticated user's assigned role.
10.	FR-010	The system shall display a Registration Dashboard with navigation links and summary cards based on the logged-in user's role.
11.	FR-011	The system shall hide node-related dashboard information when the user's role is not allowed to access node management.
12.	FR-012	The system shall hide asset-related dashboard information when the user's role is not allowed to access asset management.
13.	FR-013	The system shall allow authorized users to open the Node Registration page.
14.	FR-014	The system shall retrieve and display node records, including discovered, assigned, online, offline, disabled, and unknown nodes.
15.	FR-015	The system shall display node details, including device name, role, status, hospital, room, and the last ping time.
16.	FR-016	The system shall allow authorized users to assign an eligible node to a hospital, node role, alias, and room when required.
17.	FR-017	The system shall allow authorized users to edit the assignment data for an already-assigned node.
18.	FR-018	The system shall allow authorized users to send a blink command to an assigned online node for physical identification.
19.	FR-019	The system shall publish node blink commands through MQTT and process node acknowledgment messages when available.
20.	FR-020	The system shall allow authorized users to unassign a node after validating that the node can be safely unassigned.

21.	FR-021	The system shall allow authorized users to delete eligible node records after confirmation.
22.	FR-022	The system shall allow authorized users to open the Asset Registration page and view registered asset data.
23.	FR-023	The system shall display asset information, including asset name, RFID tag ID, status, last known location, and available actions.
24.	FR-024	After validating a registration-scan message, the system shall make the scanned RFID tag ID available to the Asset Registration workflow.
25.	FR-025	The system shall display the latest scanned RFID tag ID on the Asset Registration page.
26.	FR-026	The system shall allow authorized users to register a new asset using an RFID tag ID, asset name, and assigned room.
27.	FR-027	The system shall prevent registration of an RFID tag that is already registered as an active asset.
28.	FR-028	The system shall mark a newly registered asset as pending placement until it is detected in its assigned room.
29.	FR-029	The system shall allow authorized users to deregister an existing active asset.
30.	FR-030	The system shall update a deregistered asset, so it is no longer treated as an actively tracked hospital asset.
31.	FR-031	The system shall allow monitor-side users to request asset movement by selecting an active asset and a destination room.
32.	FR-032	The system shall reject duplicate movement requests when another pending request already exists for the selected asset.
33.	FR-033	The system shall allow monitor-side users to cancel a pending movement request.
34.	FR-034	The system shall prevent cancellation of movement requests that have already been approved, rejected, or completed.
35.	FR-035	The system shall display pending movement requests to authorized registration-side users.
36.	FR-036	The system shall allow authorized registration-side users to approve a pending movement request.
37.	FR-037	The system shall set the asset movement state to in transit after a movement request is approved.
38.	FR-038	The system shall allow authorized registration-side users to reject a pending movement request.
39.	FR-039	The system shall update the asset movement state after a movement request is rejected.
40.	FR-040	The system shall complete an approved movement when the asset is detected at the approved destination checkpoint node.
41.	FR-041	The system shall update the assigned room, current room, and flow status for the asset after a movement is completed.
42.	FR-042	The system shall display a Monitor Dashboard that includes assets, nodes, and a warning summary.
43.	FR-043	The system shall allow monitoring users to view each asset's current location, assigned room, status, and movement state.
44.	FR-044	The system shall allow monitoring users to view node availability and node status in the Monitor App.
45.	FR-045	The system shall allow monitoring users to view system alerts, including alert category, severity, message, and time.
46.	FR-046	The system shall allow monitoring users to view activity logs showing important assets and node events.
47.	FR-047	The system shall create or display an unknown tag alert when an RFID tag is detected, but no active asset matches the tag ID.
48.	FR-048	The system shall detect offline nodes when heartbeat messages are not received within the expected time.
49.	FR-049	The system shall display an offline node alert when a node is considered unavailable.
50.	FR-050	The system shall detect unauthorized movement when an asset is detected outside its expected room without a valid approved movement request.
51.	FR-051	The system shall display an unauthorized movement or wrong-location warning to monitoring users.
52.	FR-052	The backend shall subscribe to and receive registration-scan messages from the configured MQTT topics.
53.	FR-053	The system shall receive RFID detection messages from Checkpoint Nodes through MQTT.
54.	FR-054	The system shall receive heartbeat messages from Registration Desk Nodes and Checkpoint Nodes through MQTT.
55.	FR-055	The system shall use the MQTT broker to deliver scan, detection, heartbeat, and command messages between nodes and the backend.
56.	FR-056	The system shall validate incoming node messages before using them to update asset or node data.
57.	FR-057	The system shall open and maintain WebSocket connections for authenticated frontend clients.
58.	FR-058	The system shall broadcast asset, node, alert, activity, and movement updates to connected frontend clients through WebSocket.
59.	FR-059	The frontend shall update the visible tables, badges, dashboard counts, and alerts upon receiving a supported WebSocket event.
60.	FR-060	The frontend shall attempt to reconnect when the WebSocket connection is disconnected.
61.	FR-061	The frontend shall safely ignore unsupported or unknown WebSocket event types.
62.	FR-062	The backend shall reject restricted API requests when the authenticated user's role lacks the necessary permissions.
63.	FR-063	The frontend shall hide or block navigation options that are not allowed for the logged-in user's role.
64.	FR-064	The system shall allow authorized users to open the System Settings page.
65.	FR-065	The system shall retrieve current MQTT/system settings from the backend.
66.	FR-066	The system shall allow admin or authorized test users to update the MQTT/system settings.
67.	FR-067	The system shall validate setting values before saving them.
68.	FR-068	The system shall prevent unauthorized roles from changing system settings.
69.	FR-069	The system shall allow the first admin account to be created during setup when no admin account exists.

5.1. Embedded Firmware

The embedded firmware runs on the Registration Desk Node and Checkpoint Node hardware. It is responsible for local setup, RFID acquisition, MQTT connectivity, periodic status reporting, command execution support, and recovery from temporary network failures.

Table 5. 1. 1 List of functional requirements for embedded firmware.

No.	Functional Requirement ID	Verification
1.	FR-019	Integration Test
2.	FR-024	Integration Test
3.	FR-052	Integration Test
4.	FR-053	Integration Test
5.	FR-054	Integration Test
6.	FR-055	Integration Test

5.2. Backend System

The backend is the authoritative business-rule and data-processing layer. It authenticates users, enforces roles, validates API and MQTT inputs, coordinates node and asset workflows, writes persistent records, generates alerts and activity events, and broadcasts WebSocket updates.

Table 5. 2. 1 List of functional requirements for backend system.

No.	Functional Requirement ID	Verification
1.	FR-014	Integration Test
2.	FR-016	Integration Test
3.	FR-017	Integration Test
4.	FR-020	Integration Test
5.	FR-021	Integration Test
6.	FR-027	Unit Test & User Acceptance Test
7.	FR-030	Integration Test
8.	FR-032	Unit Test & User Acceptance Test
9.	FR-034	Unit Test & User Acceptance Test
10.	FR-037	Integration Test
11.	FR-039	Integration Test
12.	FR-040	Unit Test & User Acceptance Test
13.	FR-041	Integration Test
14.	FR-047	Integration Test
15.	FR-048	Unit Test & User Acceptance Test
16.	FR-049	Integration Test
17.	FR-050	Unit Test & User Acceptance Test
18.	FR-056	Unit Test & User Acceptance Test
19.	FR-057	Integration Test
20.	FR-058	Integration Test
21.	FR-062	Unit Test & User Acceptance Test
22.	FR-065	Integration Test
23.	FR-067	Unit Test & User Acceptance Test

5.3. Registration App

The Registration App provides role-based workflows for system setup, dashboard access, node management, asset registration and deregistration, movement decisions, password changes, user management, and settings. Business rules are still enforced by the backend, even when the frontend hides or turns off an unavailable action.

Table 5. 3. 1 List of functional requirements for registration app.

No.	Functional Requirement ID	Verification
1.	FR-010	Demonstration & User Acceptance Test
2.	FR-011	Integration Test
3.	FR-012	Integration Test
4.	FR-013	Demonstration & User Acceptance Test
5.	FR-015	Integration Test
6.	FR-018	Demonstration & User Acceptance Test
7.	FR-022	Demonstration & User Acceptance Test

8.	FR-023	Integration Test
9.	FR-025	Integration Test
10.	FR-026	Demonstration & User Acceptance Test
11.	FR-028	Integration Test
12.	FR-029	Demonstration & User Acceptance Test
13.	FR-035	Integration Test
14.	FR-036	Demonstration & User Acceptance Test
15.	FR-038	Demonstration & User Acceptance Test

5.4. Monitor App

The Monitor App provides operational visibility into asset location and flow state, node availability, alerts, and activity. It also allows authorized users to request or cancel asset movements, with the backend handling validation and persistence.

Table 5. 4. 1 List of functional requirements for monitor app.

No.	Functional Requirement ID	Verification
1.	FR-031	Demonstration & User Acceptance Test
2.	FR-033	Demonstration & User Acceptance Test
3.	FR-042	Demonstration & User Acceptance Test
4.	FR-043	Demonstration & User Acceptance Test
5.	FR-044	Demonstration & User Acceptance Test
6.	FR-045	Demonstration & User Acceptance Test
7.	FR-046	Demonstration & User Acceptance Test
8.	FR-051	Integration Test

5.5. User and Access Management

User and access management covers first-admin setup, authentication, logout, password changes, account administration, and role enforcement. Frontend restrictions are usability controls; backend authorization remains mandatory for every protected operation.

Table 5. 5. 1 List of functional requirements for user and access management.

No.	Functional Requirement ID	Verification
1.	FR-001	Demonstration & User Acceptance Test
2.	FR-002	Integration Test
3.	FR-003	Integration Test
4.	FR-004	Demonstration & User Acceptance Test
5.	FR-005	Demonstration & User Acceptance Test
6.	FR-006	Integration Test
7.	FR-007	Demonstration & User Acceptance Test
8.	FR-008	Unit Test & User Acceptance Test
9.	FR-009	Unit Test & User Acceptance Test
10.	FR-062	Unit Test & User Acceptance Test
11.	FR-063	Unit Test & User Acceptance Test
12.	FR-069	Demonstration & User Acceptance Test

5.6. System Settings

System settings allow authorized users to view and change permitted runtime configuration values. Sensitive secrets should remain within a protected deployment configuration and must not be exposed via standard UI fields.

Table 5. 6. 1 List of functional requirements for system settings.

No.	Functional Requirement ID	Verification
1.	FR-064	Demonstration & User Acceptance Test
2.	FR-065	Integration Test
3.	FR-066	Demonstration & User Acceptance Test
4.	FR-067	Unit Test & User Acceptance Test
5.	FR-068	Unit Test & User Acceptance Test

5.7. Cross-System Workflows

The following workflows cross firmware, MQTT, backend, database, WebSocket, and frontend boundaries. They require coordinated end-to-end verification because a single subsystem test cannot confirm successful behavior.

Table 5. 7. 1 List of expected results from the integrated system workflow.

No.	Workflow	Primary Frequency Requirement IDs	Expected End-to-End Result
1.	First-admin setup	- FR-069 - FR-002 - FR-003	When no admin exists, the setup page creates one active admin and then becomes unavailable.
2.	Node identification	- FR-018 - FR-019 - FR-055	A permitted user sends a blink command; the target node blinks, and an acknowledgment or clear failure status is returned.
3.	Asset registration	- FR-024 - FR-026 - FR-027 - FR-028 - FR-052 - FR-058 - FR-059	A registration scan reaches the backend, the user registers an asset, and all relevant views update.
4.	Initial Placement	- FR-028 - FR-053 - FR-041 - FR-058 - FR-059	A newly registered asset is placed after a valid detection in its assigned room.
5.	Movement request and decision	- FR-031 - FR-032 - FR-033 - FR-034 - FR-035 - FR-036 - FR-037 - FR-038 - FR-039	A request is created, displayed, approved, or rejected once and propagated to both applications.
6.	Movement completion	- FR-040 - FR-041 - FR-053 - FR-058 - FR-059	Detection at the approved destination completes the movement and updates the assigned and current room.
7.	Unauthorized movement	- FR-050 - FR-051 - FR-053 - FR-058 - FR-059	A detection outside the expected workflow produces a warning without incorrectly completing movement.
8.	Offline-node detection	- FR-048 - FR-049 - FR-054 - FR-058 - FR-059	Missing heartbeats cause the node to become offline, and an alert is visible.
9.	Live update recovery	- FR-057 - FR-058 - FR-059 - FR-060 - FR-061	Clients reconnect after interruption and continue processing supported events without manual refresh.

6. External Interface Requirements

External interfaces define how users, hardware, software components, and communication services interact with the product. Interface definitions shall be sufficiently precise to support independent implementation and integration testing.

6.1. Hardware Interfaces

Hardware interfaces cover the ESP32 controller, UHF RFID reader, antenna, power supply, status indicators, wiring, enclosure, reset, and configuration controls. Electrical compatibility and installation safety shall be in accordance with the selected hardware documentation and site test results.

Table 6. 1. 1 List of hardware interface requirements.

No.	ID	Hardware Interface Requirement
1.	HWI-001	The node shall use an ESP32-compatible controller with Wi-Fi support and sufficient memory for the selected firmware modules.
2.	HWI-002	The node shall provide a documented electrical and communication interface compatible with the selected UHF RFID reader.
3.	HWI-003	The power supply shall provide the voltage and peak current required for simultaneous operation of the ESP32, RFID reader, Wi-Fi, and LED.
4.	HWI-004	The node shall provide red and green visual indicators for supported status and identification behavior.
5.	HWI-005	The node shall provide a supported reset or configuration mechanism accessible during installation and maintenance.
6.	HWI-006	RFID antenna placement and cabling shall follow the reader datasheet and site-specific installation test results.
7.	HWI-007	Connectors and wiring shall be mechanically secure and protected from accidental short circuits or disconnection.
8.	HWI-008	The enclosure and installation method shall prevent exposed conductive parts and shall not obstruct normal hospital operations.

The table above explains how the node module hardware provides the interface to users and maintainers. The pins and wiring for each part of the module listed in that table are shown in the wiring diagram below. Details are explained in the Project System Design Document.

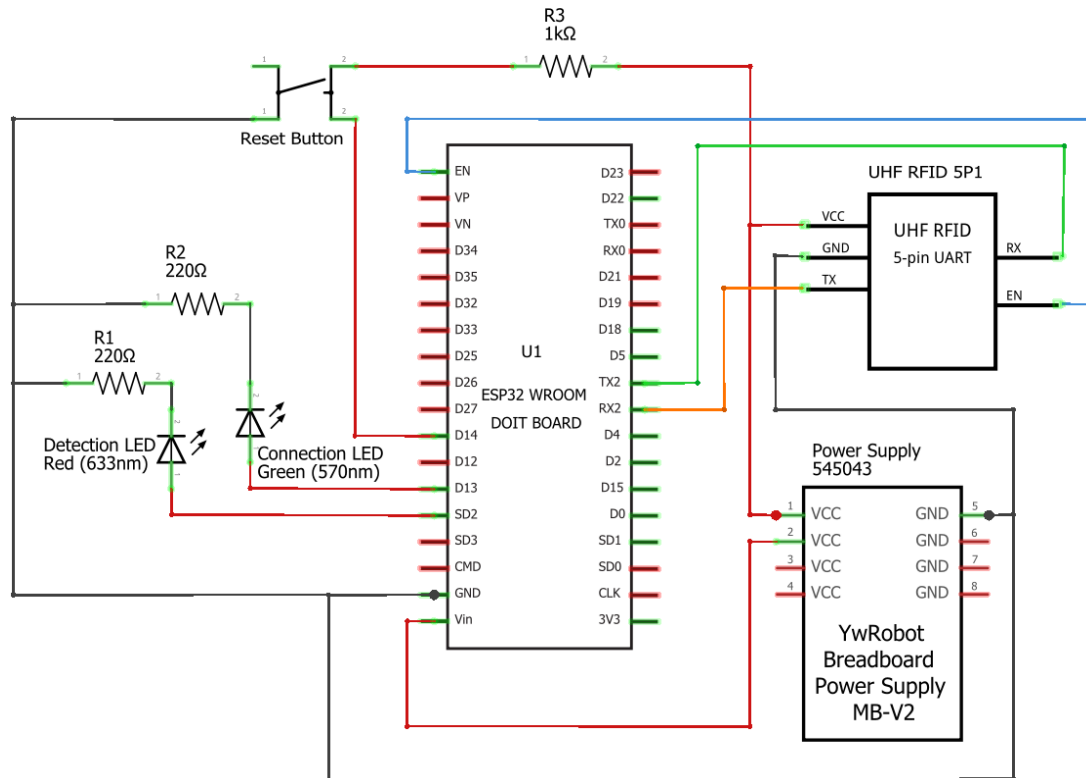


Figure 6. 1. 1 Sensor Node Hardware Wiring Diagram.

6.2. Software and API Interfaces

Software interfaces define the authenticated web APIs, database access, environment configuration, documentation policy, frontend build configuration, migration behavior, and compatibility expectations. Public contracts shall use structured, machine-readable fields rather than depending on raw exception text.

Table 6. 2. 1 List of software interface requirements.

No.	ID	Interface	Requirement
1.	SWI-001	Frontend to Backend API	The Registration App and Monitor App shall use HTTPS JSON APIs for authenticated reads and commands.
2.	SWI-002	Authentication	Protected API requests shall include a valid bearer token and shall be authorized by backend role checks.
3.	SWI-003	Backend to Database	The backend shall access PostgreSQL through the configured SQLAlchemy connection and managed migrations.
4.	SWI-004	Backend API Responses	API responses shall use consistent status codes and machine-readable error details.
5.	SWI-005	Environment Configuration	Deployment-specific URLs, credentials, origins, and feature flags shall be configurable without source-code modification.
6.	SWI-006	API Documentation	Interactive API documentation shall be turned off or restricted in production, in accordance with the deployment policy.
7.	SWI-007	Frontend Build Configuration	Each frontend shall support separate development and production API and WebSocket endpoints.
8.	SWI-008	Database Migration	Schema changes shall be applied through version-controlled migrations before incompatible application code is activated.
9.	SWI-009	Error Contract	The frontend shall not depend on unstructured exception text when a stable error code or detail field is available.
10.	SWI-010	Compatibility	Changes to public API fields or event names shall be versioned or coordinated across affected clients.

6.3. Communication Interfaces

The system uses HTTPS with JSON for request-response operations, WSS/WebSocket for backend-initiated live updates, MQTT for node-to-backend events and backend-to-node commands, UART for RFID-reader communication, GPIO for local indicators, and PostgreSQL connections for persistence. Each hospital or deployment shall use an isolated MQTT namespace or an equivalent coordination mechanism.

Table 6. 3. 1 List of system communication interface requirements.

No.	ID	Communication Requirement
1.	COM-001	Production HTTP traffic shall use HTTPS.
2.	COM-002	Production WebSocket traffic shall use secure WebSocket connections where supported by the deployment environment.
3.	COM-003	MQTT topic structures shall identify the hospital or deployment namespace, node, and message purpose.
4.	COM-004	Node messages shall include the fields required to identify the node, the event type, the relevant tag ID, and, when applicable, the event time.
5.	COM-005	The backend shall validate the MQTT topic context and payload content before updating persistent data.
6.	COM-006	The firmware shall automatically reconnect to Wi-Fi and MQTT after a temporary disconnection.
7.	COM-007	The firmware shall resubscribe to required command topics after MQTT reconnection.
8.	COM-008	Supported command messages shall produce an acknowledgment or a clear timeout/failure status.
9.	COM-009	WebSocket events shall use stable event-type names and structured payloads.
10.	COM-010	Frontend clients shall ignore unknown WebSocket events without crashing.
11.	COM-011	The web applications shall not connect directly to MQTT.
12.	COM-012	Connection and reconnection behavior shall avoid unbounded rapid retry loops.

The communication requirements above shall be used by the system in layers, as shown in the image below. The system uses three layers of communication: the Web Application layer, Sensor Node layer, and Backend layer. The Web Application layer interacts directly with users via the UI and with the system remotely over HTTPS via WebSocket. The Sensor Node layer communicates with the backend via an MQTT broker over the Wi-Fi network and receives user triggers from its sensor. And the Backend layer bridges the Sensor Node and the Web Application via MQTT and WebSocket over the Wi-Fi network, and also directly interacts with the system database. The diagram below is only an overview of the overall communication flow within the Fixed-Asset Tracker System Product. The details about it are explained in the System Design Document.

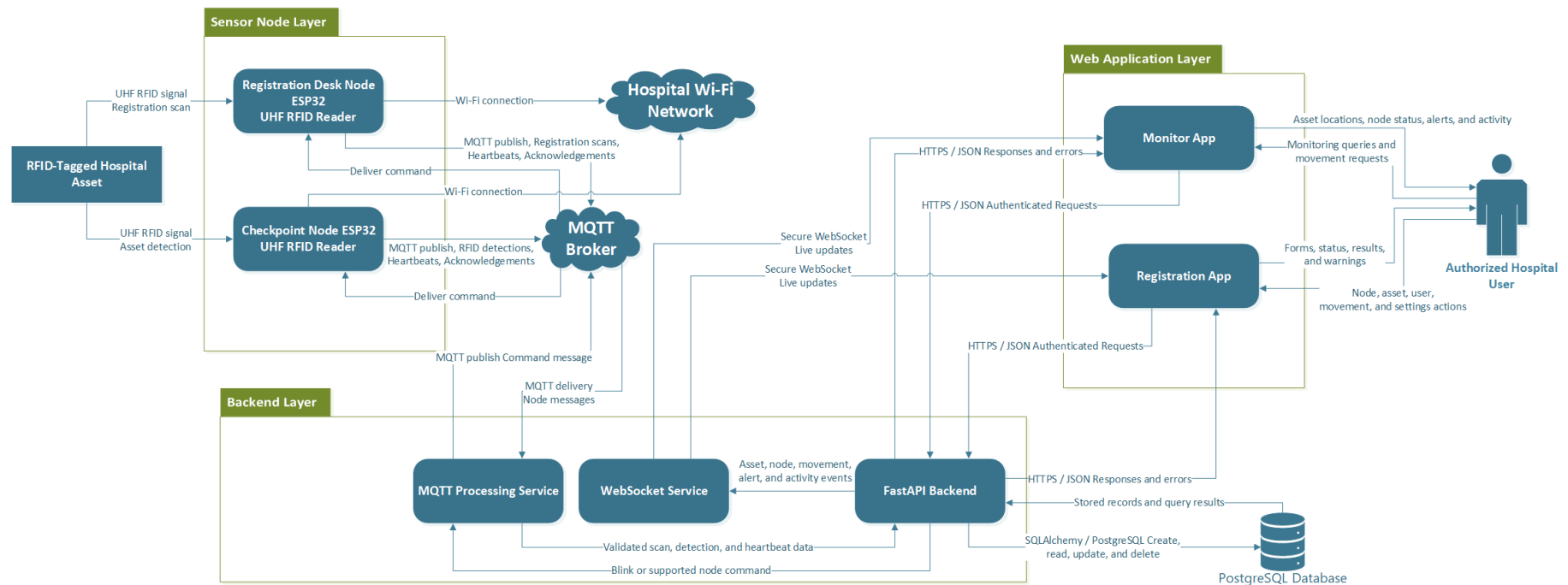


Figure 6. 3. 1 System Communication Flow Diagram.

6.4. User Interface Requirements

User-interface requirements define role-based visibility, loading and error states, accessible status communication, table behavior, modal interaction, responsive layout, form labeling, password controls, connection visibility, and time display. Critical meaning shall never rely only on color.

Table 6. 4. 1 List of user interface requirements.

No.	ID	User Interface Requirement
1.	UI-001	Protected pages shall show only navigation options allowed for the current role.
2.	UI-002	The interface shall display clear loading, success, error, empty, offline, and access-denied states.
3.	UI-003	Asset, node, alert, and movement states shall use consistent labels and visual badges across both applications.
4.	UI-004	Critical state information shall not rely solely on color; text labels or icons shall also be provided.
5.	UI-005	Tables shall support appropriate search, sorting, filtering, and pagination functions.
6.	UI-006	Modal dialogs shall support clear close actions and keyboard dismissal where appropriate.
7.	UI-007	Disabled actions shall provide sufficient context when the user cannot perform them.
8.	UI-008	The layout shall remain usable on common desktop, half-window, tablet, and supported mobile widths.
9.	UI-009	Form fields shall have visible labels, validation messages, and sensible keyboard focus order.
10.	UI-010	Password visibility controls shall use accessible labels and shall not change the stored password value.
11.	UI-011	The live connection status shall be visible when a workflow depends on the availability of APIs or WebSockets.
12.	UI-012	User-facing timestamps shall be displayed in the configured local time zone and format.

7. Data Requirements

Data requirements define the principal entities, integrity rules, time handling, and recovery expectations. The implementation schema may evolve, but it shall preserve the business meanings and constraints stated here.

7.1. Main Data Entities

The following entities describe the principal persistent business data. They may be implemented using PostgreSQL tables, SQLAlchemy models, and related API schemas. The hospital and Room are included because they appear in the logical data model and are referenced by nodes, assets, and movement requests.

Table 7. 1. 1 List of data requirements for system entities and their important data.

No.	Entity	Purpose	Important Data
1.	User	Represents an authenticated person and role.	ID, name, username/email, password hash, role, active status, timestamps.
2.	Node	Represents a Registration Desk or Checkpoint Node.	Device ID, alias, role, status, hospital, room, last ping, assignment data.
3.	Asset	Represents an RFID-tagged hospital asset.	Asset ID, tag ID, name, active status, assigned room, current room, last seen, flow status.
4.	Movement Request	Represents the requested and decided asset relocation.	Request ID, asset, requester, source, destination, status, approver, timestamps.
5.	Detection Event	Records a node detecting an RFID tag.	Tag ID, node ID, event time, result, and asset reference when known.
6.	Alert	Represents a system warning requiring user attention.	Type, severity, message, related asset/node, status, timestamps.
7.	Activity Event	Provides an operational history of significant actions and state changes.	Event type, actor, related entity, details, timestamp.
8.	System Setting	Stores permitted runtime configuration values.	Key, value, type, validation constraints, updated by, updated time.

The entities above are similar to the system object class in object-oriented programming. However, these entities do not specifically show the object members and functions. They include only the object attributes that can later be used as object members in the class diagram.

The entity's relationship overview is shown in the diagram below. The detailed versions of each layer can be found in the System Design Document.

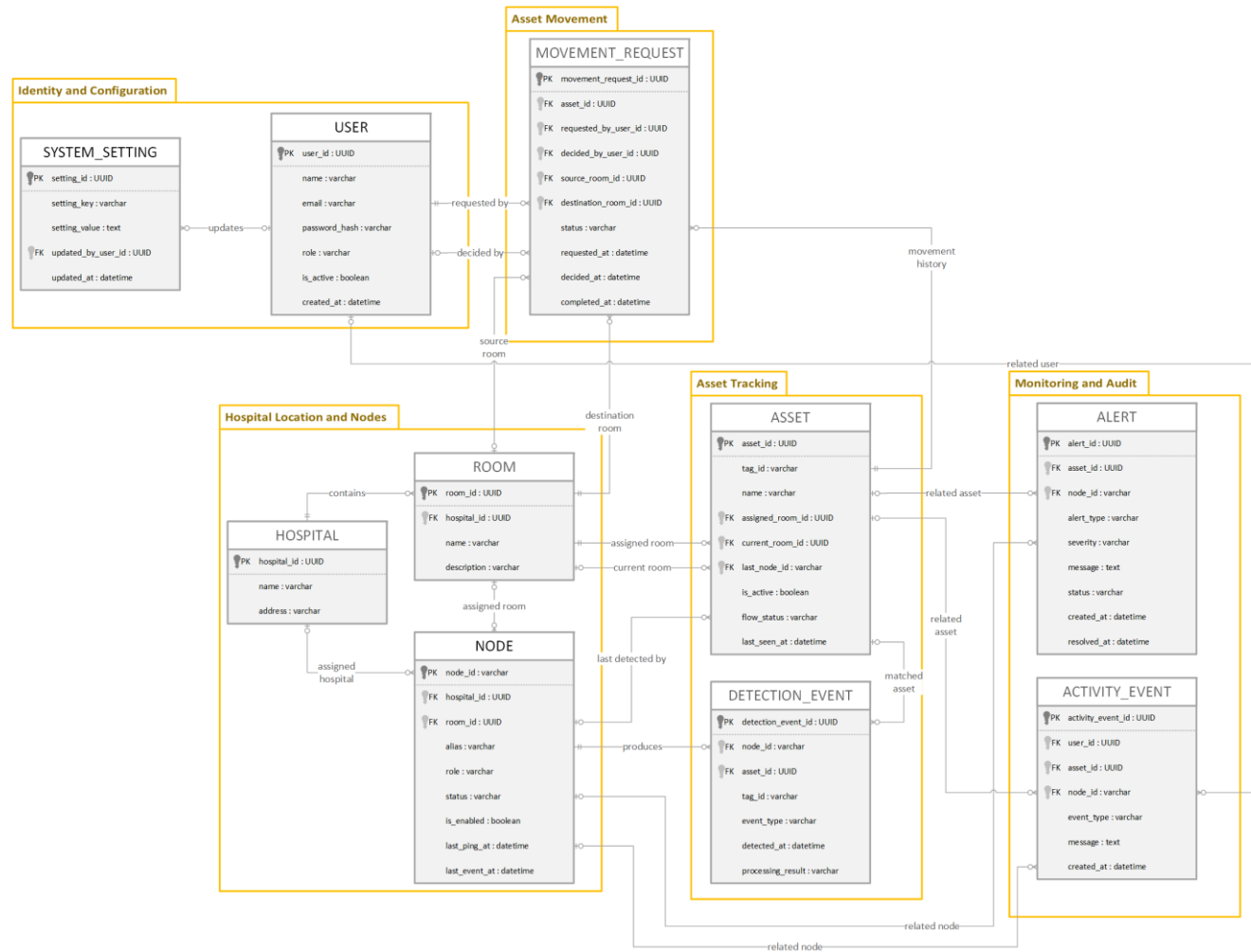


Figure 7. 1. 1 Entity-Relationship Diagram.

7.2. Data Validation and Integrity

Validation and integrity rules protect the consistency of tags, nodes, rooms, assets, movement requests, user roles, enumerations, MQTT messages, and multi-record database updates. Backend validation is authoritative even when frontend validation is also provided.

Table 7. 2. 1 List of data requirements for system data validation.

No.	ID	Data Requirement
1.	DATA-001	Each active RFID tag ID shall identify no more than one active asset.
2.	DATA-002	Each node shall have a unique device identifier.
3.	DATA-003	An assigned Checkpoint Node shall reference a valid hospital and room.
4.	DATA-004	A Registration Desk Node shall not require a normal checkpoint room assignment when the role rule excludes it.
5.	DATA-005	A pending movement request shall reference one active asset and one valid destination.
6.	DATA-006	The system shall prevent more than one active pending movement request for the same asset.
7.	DATA-007	A movement shall not be completed unless the detected destination matches the approved destination.
8.	DATA-008	The system shall preserve consistency between the assigned room, current room, movement status, and asset flow status.
9.	DATA-009	Deregistered assets shall not be treated as active tracked assets.
10.	DATA-010	Unknown RFID tags may produce detection and alert records without automatically creating an active asset.
11.	DATA-011	User roles shall be limited to the configured allowed role values.
12.	DATA-012	The system shall prevent deactivation or demotion of the last active admin.
13.	DATA-013	Status values shall use defined enumerations rather than unrestricted free text.
14.	DATA-014	Invalid or incomplete MQTT payloads shall not modify asset or node records.
15.	DATA-015	Database changes that affect related entities shall use transactional behavior to prevent partial updates from creating inconsistencies.

7.3. Time and Date Handling

Time requirements ensure that stored timestamps, node events, heartbeat timeout calculations, alerts, and user-facing displays use compatible time bases. The backend should store canonical UTC timestamps and convert them to the configured local timezone for display.

Table 7. 3. 1 List of data requirements for system date and time formatting.

No.	ID	Time and Date Requirement
1.	TIME-001	The backend shall use a consistent canonical time basis for stored timestamps, preferably UTC.
2.	TIME-002	User-facing timestamps shall be converted to the configured local time zone, including WIB where applicable.
3.	TIME-003	Heartbeat timeout calculations shall compare timestamps expressed on the same time basis.
4.	TIME-004	Incoming node timestamps shall be validated or normalized before they are used for ordering and timeout logic.
5.	TIME-005	Activity and alert records shall preserve sufficient timestamp precision to reconstruct the order of events.

7.4. Retention, Backup, and Recovery

Retention, backup, and recovery requirements protect operational continuity and historical traceability. Final retention periods, backup frequency, Recovery Point Objective, and Recovery Time Objective remain deployment decisions under control and shall be approved before production acceptance.

Table 7. 4. 1 List of data requirements for system recovery.

No.	ID	Retention / Recovery Requirement
1.	REC-001	Active user, node, asset, and movement data shall be retained as long as required for normal operation.
2.	REC-002	Activity, alert, and detection-event retention periods shall be defined before production deployment. Current target: every 29 days for data migrations.
3.	REC-003	The production database shall use scheduled backups appropriate to the accepted recovery point objective. Target frequency: once a month, preferably every 29 days, before or after the data migrations.
4.	REC-004	A documented database restoration procedure shall be tested before production acceptance.
5.	REC-005	Deployment configuration and database credentials shall be stored separately from backup data.
6.	REC-006	Database schema migrations shall be version-controlled and recoverable.
7.	REC-007	Expired or replaced development databases shall be migrated or recreated in accordance with documented procedures.
8.	REC-008	Deletion and deregistration behavior shall distinguish between permanent removal and operational deactivation when historical traceability is required.

8. Non-Functional Requirements

Non-functional requirements define the product's quality attributes and operational constraints. Requirements using “shall” are mandatory. Requirements using “should” are intended targets that require final measurable thresholds or deployment decisions before formal acceptance.

As with the functional requirements, the non-functional requirements are retained to remain consistent with the companion Use Case Specification and its traceability matrices. Each requirement shall be verified using inspection, demonstration, unit testing, integration testing, or end-to-end testing as indicated.

The table below defines the non-functional requirements of the Hospital Fixed-Asset Tracking System product with their categories. The subsections below explain each category, including descriptions of the non-functional requirements and their verification.

Table 8. 1 List of non-functional system requirements.

No.	ID	Category
1.	NFR-001	Security
2.	NFR-002	Security
3.	NFR-003	Security
4.	NFR-004	Security
5.	NFR-005	Security
6.	NFR-006	Security
7.	NFR-007	Security
8.	NFR-008	Security
9.	NFR-009	Reliability
10.	NFR-010	Reliability
11.	NFR-011	Reliability
12.	NFR-012	Reliability
13.	NFR-013	Reliability
14.	NFR-014	Availability
15.	NFR-015	Availability
16.	NFR-016	Performance
17.	NFR-017	Performance
18.	NFR-018	Performance
19.	NFR-019	Usability
20.	NFR-020	Usability
21.	NFR-021	Usability
22.	NFR-022	Usability
23.	NFR-023	Usability
24.	NFR-024	Usability
25.	NFR-025	Maintainability
26.	NFR-026	Maintainability
27.	NFR-027	Maintainability
28.	NFR-028	Maintainability
29.	NFR-029	Compatibility
30.	NFR-030	Compatibility
31.	NFR-031	Data Integrity
32.	NFR-032	Data Integrity
33.	NFR-033	Data Integrity
34.	NFR-034	Data Integrity
35.	NFR-035	Auditability
36.	NFR-036	Scalability

8.1. Security and Privacy

This product is intended to track assets rather than patients. Personal data collection shall be limited to the minimum account information needed for authentication, authorization, accountability, and administration.

Table 8. 1. 1 List of non-functional requirements for security and privacy.

No.	ID	Non-Functional Requirement Description	Verification
1.	NFR-001	The system shall require users to log in before accessing protected pages in the Registration App or Monitor App.	Test/analysis
2.	NFR-002	The system shall restrict page access and actions based on the authenticated user's assigned role.	Test/analysis
3.	NFR-003	The system shall store user passwords securely using password hashing instead of storing plain-text passwords.	Test/analysis
4.	NFR-004	The system shall reject login attempts from disabled user accounts.	Test/analysis
5.	NFR-005	The system shall prevent unsafe admin account actions, such as disabling or demoting the last active admin account.	Test/analysis
6.	NFR-006	The system shall clear local authentication data when the user logs out or when the session is no longer valid.	Test/analysis
7.	NFR-007	The system shall be deployed over HTTPS to ensure that authentication data and API communication are transmitted securely.	Test/analysis
8.	NFR-008	For production deployment, the system shall avoid exposing sensitive configuration values, such as provisioning secrets, database credentials, and production tokens, in public source code.	Test/analysis

8.2. Reliability and Availability

Reliability requirements define graceful behavior for temporary service interruptions, input validation, offline node detection, and unsupported events. Availability targets remain deployment decisions and shall be measurable before formal acceptance.

Table 8. 2. 1 List of non-functional requirements for system reliability.

No.	ID	Non-Functional Requirement Description	Verification
1.	NFR-009	The system shall continue to display the most recent stored asset and node data even when live WebSocket updates are temporarily unavailable.	Test/analysis
2.	NFR-010	The frontend shall attempt to reconnect when the WebSocket connection is interrupted.	Test/analysis
3.	NFR-011	The system shall detect offline nodes when expected heartbeat messages are not received within the configured timeout period.	Test/analysis
4.	NFR-012	The system shall validate incoming MQTT messages before using them to update asset or node data.	Test/analysis
5.	NFR-013	The system shall handle unknown RFID tags, invalid nodes, and unsupported WebSocket event types without crashing the application.	Test/analysis
6.	NFR-014	The backend API should remain available for normal registration, monitoring, and movement workflows during expected operating hours.	Test/analysis
7.	NFR-015	If the backend, database, MQTT broker, or WebSocket service is unavailable, the frontend shall show a clear error or connection status message to the user.	Test/analysis

8.3. Performance

Performance requirements cover initial page load, live update delay, and usability of table operations. The final response-time and event-delay targets shall be documented in Appendix C before formal production acceptance.

Table 8. 3. 1 List of non-functional requirements for system performance.

No.	ID	Non-Functional Requirement Description	Verification
1.	NFR-016	The system should display page data within a reasonable loading time under normal network conditions.	Test/analysis
2.	NFR-017	The system should update the frontend asset and node status shortly after the backend receives a valid MQTT or movement event.	Test/analysis
3.	NFR-018	The system should support searching, sorting, filtering, and pagination without making asset and node tables difficult to use.	Test/analysis

8.4. Usability and Accessibility

Usability and accessibility requirements ensure that users can understand status, complete critical workflows, receive clear feedback, and operate the applications across supported screen sizes without requiring technical knowledge of MQTT, WebSocket, or database behavior.

Table 8. 4. 1 List of non-functional requirements for system usability.

No.	ID	Non-Functional Requirement Description	Verification
1.	NFR-019	The user interface shall clearly show the current status of assets, nodes, alerts, and movement requests.	Test/analysis
2.	NFR-020	The system shall use clear visual indicators, such as badges, colors, and warning states, to help users identify important conditions of assets and nodes.	Test/analysis
3.	NFR-021	The system shall provide clear success, error, loading, and empty-state messages for important user actions.	Test/analysis
4.	NFR-022	The system shall hide or turn off actions that the current user role is not allowed to perform.	Test/analysis
5.	NFR-023	The application layout should remain usable on common desktop, half-window, tablet, and mobile screen sizes.	Test/analysis
6.	NFR-024	Important workflows, such as registering assets, assigning nodes, requesting movement, and approving movement, should be understandable without requiring technical knowledge of MQTT, WebSocket, or database behavior.	Test/analysis

8.5. Scalability

Scalability requirements ensure that the architecture can support additional checkpoint nodes, tracked assets, users, and event volume without major redesign. Numerical capacity targets remain TBD and shall be verified using representative load tests.

Table 8. 5. 1 Non-functional requirements for system scalability.

No.	ID	Non-Functional Requirement Description	Verification
1.	NFR-036	The system should be designed so that additional checkpoint nodes and tracked assets can be added without requiring major changes to the system architecture.	Test/analysis

8.6. Maintainability and Testability

Maintainability and testability requirements promote small, focused modules, reusable services, consistent naming, automated tests, and clear operational records. These requirements reduce the cost and risk of future modification.

Table 8. 6. 1 List of non-functional requirements for system maintainability.

No.	ID	Non-Functional Requirement Description	Verification
1.	NFR-025	The frontend, backend, and firmware code should be organized into small, readable modules to support future maintenance.	Test/analysis
2.	NFR-026	Reusable logic should be placed in services, hooks, utilities, or helper modules rather than duplicated across pages.	Test/analysis
3.	NFR-027	The system should use consistent naming for roles, asset statuses, node statuses, movement states, and event types.	Test/analysis
4.	NFR-028	The system should include tests for critical backend workflows, such as authentication, role access, node management, asset registration, and asset movement.	Test/analysis

8.7. Compatibility and Portability

Compatibility and portability requirements define supported browser families and the selected ESP32, RFID, Wi-Fi, and MQTT environment. Deployment-specific configuration shall remain separate from source code where practical.

Table 8. 7. 1 List of non-functional requirements for system compatibility.

No.	ID	Non-Functional Requirement Description	Verification
1.	NFR-029	The web applications should run on modern desktop browsers such as Chrome, Edge, and Firefox.	Test/analysis
2.	NFR-030	The sensor node firmware should support the selected ESP32-based hardware, RFID reader, Wi-Fi connection, and MQTT communication setup.	Test/analysis

8.8. Data Integrity

Data-integrity requirements ensure that duplicate tags, room assignments, movement state, asset flow status, and node status remain consistent after registration, deregistration, movement decisions, detections, and heartbeats.

Table 8. 8. 1 List of non-functional requirements for data integrity.

No.	ID	Non-Functional Requirement Description	Verification
1.	NFR-031	The system shall prevent duplicate active registration of the same RFID tag.	Test/analysis
2.	NFR-032	The system shall preserve consistency between the assigned asset room, the current room, the movement request state, and the flow status.	Test/analysis
3.	NFR-033	The system shall not complete an asset movement unless the asset is detected at the approved destination checkpoint node.	Test/analysis
4.	NFR-034	The system shall keep asset and node status records consistent after registration, deregistration, movement approval, movement rejection, heartbeat updates, and RFID detections.	Test/analysis

8.9. Auditability

Auditability requires the system to preserve understandable records of significant state changes and user actions without exposing secrets or unnecessary technical details.

Table 8. 9. 1 Non-functional requirement for system auditability.

No.	ID	Non-Functional Requirement Description	Verification
5.	NFR-035	The system should maintain activity records for important events, including node status changes, asset detections, movement requests, approvals, rejections, and warnings.	Test/analysis

9. Safety, Privacy, and Compliance Requirements

The product operates in a hospital environment but is not intended to make clinical decisions. Formal regulatory specification and organization-specific compliance obligations shall be confirmed before the production deployment.

Those confirmations and obligations are listed as follows:

9.1. List of Safety Requirements

Safety requirements ensure that system failure does not prevent clinical work and that sensor-node installation, electrical connections, antennas, cables, enclosures, and user-visible location statements do not introduce avoidable operational or physical hazards.

Table 9. 1 List of safety requirements.

No.	ID	Requirement
1.	SAFE-001	Failure of the tracking system shall not prevent hospital staff from continuing normal clinical operations.
2.	SAFE-002	The product shall not diagnose patients, recommend treatment, or automatically control medical equipment.
3.	SAFE-003	Sensor nodes, antennas, cables, and power supplies shall be installed so as not to create accessible electrical hazards, trip hazards, or obstruction hazards.
4.	SAFE-004	The installation process shall avoid interfering with medical device operation and shall follow the hospital site approval process.
5.	SAFE-005	The system shall not represent an asset location as guaranteed when the relevant node is offline, or communication is unavailable.
6.	SAFE-006	User-visible warnings shall clearly distinguish tracking uncertainty from confirmed asset location.

9.2. List of Privacy Requirements

Privacy requirements limit collected data to account and asset information required for operation. Patient identifiers and clinical records remain outside the intended scope, and protected operational data requires authenticated and authorized access.

Table 9. 2 List of privacy requirements.

No.	ID	Requirement
1.	PRIV-001	The system shall collect only the user and asset information required for asset-tracking operations.
2.	PRIV-002	Patient identifiers and clinical records shall not be stored within the intended scope of asset tracking.
3.	PRIV-003	Protected operational data shall require authenticated and authorized access.

9.3. List of Compliance Requirements

Compliance requirements require the deployment to follow applicable hospital policies and prohibit unsupported claims of regulatory or standards compliance. Any formal claim shall be based on documented assessment and evidence.

Table 9. 3 List of compliance requirements.

No.	ID	Requirement
1.	CMP-001	Production deployment shall follow applicable hospital IT, cybersecurity, electrical, radio, privacy, and record-retention policies.
2.	CMP-002	Any claimed regulatory or standards compliance shall be supported by documented assessment and evidence rather than assumption.

10. Error Handling and Recovery Requirements

Error-handling and recovery requirements define how the frontend, backend, firmware, database, MQTT, WebSocket, movement workflows, authentication, and provisioning respond to invalid input or temporary failure. Errors shall be visible, recoverable where possible, and shall not create false success or inconsistent persistent state.

Table 10. 1 List of system error handling and recovery requirements.

No.	ID	Error Handling/Recovery Requirement
1.	ERR-001	The frontend shall display a clear message when an API request fails and shall not leave the page in an indefinite loading state.
2.	ERR-002	The backend shall return consistent error codes and details for validation, authentication, authorization, not-found, conflict, and server errors.
3.	ERR-003	Malformed MQTT messages shall be rejected without updating node or asset records.
4.	ERR-004	Unknown RFID tags shall be handled as a defined warning condition rather than causing an application failure.
5.	ERR-005	A node heartbeat timeout shall update the node to the appropriate offline state and shall not delete the node.
6.	ERR-006	The firmware shall retry Wi-Fi and MQTT connections using bounded retry or backoff behavior.
7.	ERR-007	After MQTT reconnection, the firmware shall restore required subscriptions.
8.	ERR-008	If a blink acknowledgment is not received, the user shall be shown a timeout, pending, or failure result.
9.	ERR-009	The system shall reject duplicate pending movement requests for the same asset.
10.	ERR-010	The system shall reject movement approval, rejection, cancellation, or completion when the request is no longer in the required state.
11.	ERR-011	Detection at the wrong destination shall not complete an approved movement.
12.	ERR-012	A temporary WebSocket failure shall not prevent users from manually refreshing stored data.
13.	ERR-013	Expired or invalid sessions shall redirect users to login and clear the local protected state.
14.	ERR-014	Database failures shall not return a false success result to the frontend.
15.	ERR-015	Provisioning or setup failures shall display a recoverable error and preserve the ability to retry configuration.

11. Deployment and Configuration Requirements

The system supports separate local development, test, and production environments. Production deployment shall not depend on development-only defaults.

Deployment and configuration requirements define environment variables, secure endpoints, frontend build configuration, migrations, first-admin setup, MQTT worker ownership, namespace separation, route rewrites, backup procedures, firmware provisioning, reproducible builds, and accepted hosting limitations. Production deployment shall not depend on undocumented local settings.

Table 11. 1 List of system deployment and configuration requirements.

No.	ID	Deployment/Configuration Requirement
1.	DEP-001	The backend shall support configuring the database, MQTT, allowed origins, JWT, documentation, and environment-specific settings via protected configuration or environment variables.
2.	DEP-002	Production API communication shall use HTTPS and secure WebSocket endpoints.
3.	DEP-003	The Registration App and Monitor App shall support separate development and production configurations for API and WebSocket endpoints.
4.	DEP-004	Production secrets shall not be committed to source control.
5.	DEP-005	The database schema shall be initialized and upgraded through documented migrations.
6.	DEP-006	The first admin setup shall be available only when no admin account exists.
7.	DEP-007	The production backend shall use a deployment model suitable for MQTT subscription ownership and WebSocket delivery.
8.	DEP-008	Running multiple backend instances against the same MQTT namespace shall require an explicit coordination strategy to avoid duplicate or missing live updates.
9.	DEP-009	Local and production deployments should use separate MQTT namespaces or an explicit worker-enable setting when both may run simultaneously.
10.	DEP-010	Frontend routes shall be configured so that direct navigation and page refreshes return to the application entry point.
11.	DEP-011	A deployment checklist shall define required environment variables and their allowed values.
12.	DEP-012	Database backup and restoration procedures shall be documented for the selected hosting environment.
13.	DEP-013	Sensor nodes shall provide a documented installation and provisioning procedure.
14.	DEP-014	Firmware builds shall be reproducible from version-controlled source and declared PlatformIO environments.
15.	DEP-015	A production deployment shall not rely on free-tier availability guarantees unless explicitly accepted as a project limitation.

Based on the requirements table above, the system deployment should follow the diagram below.

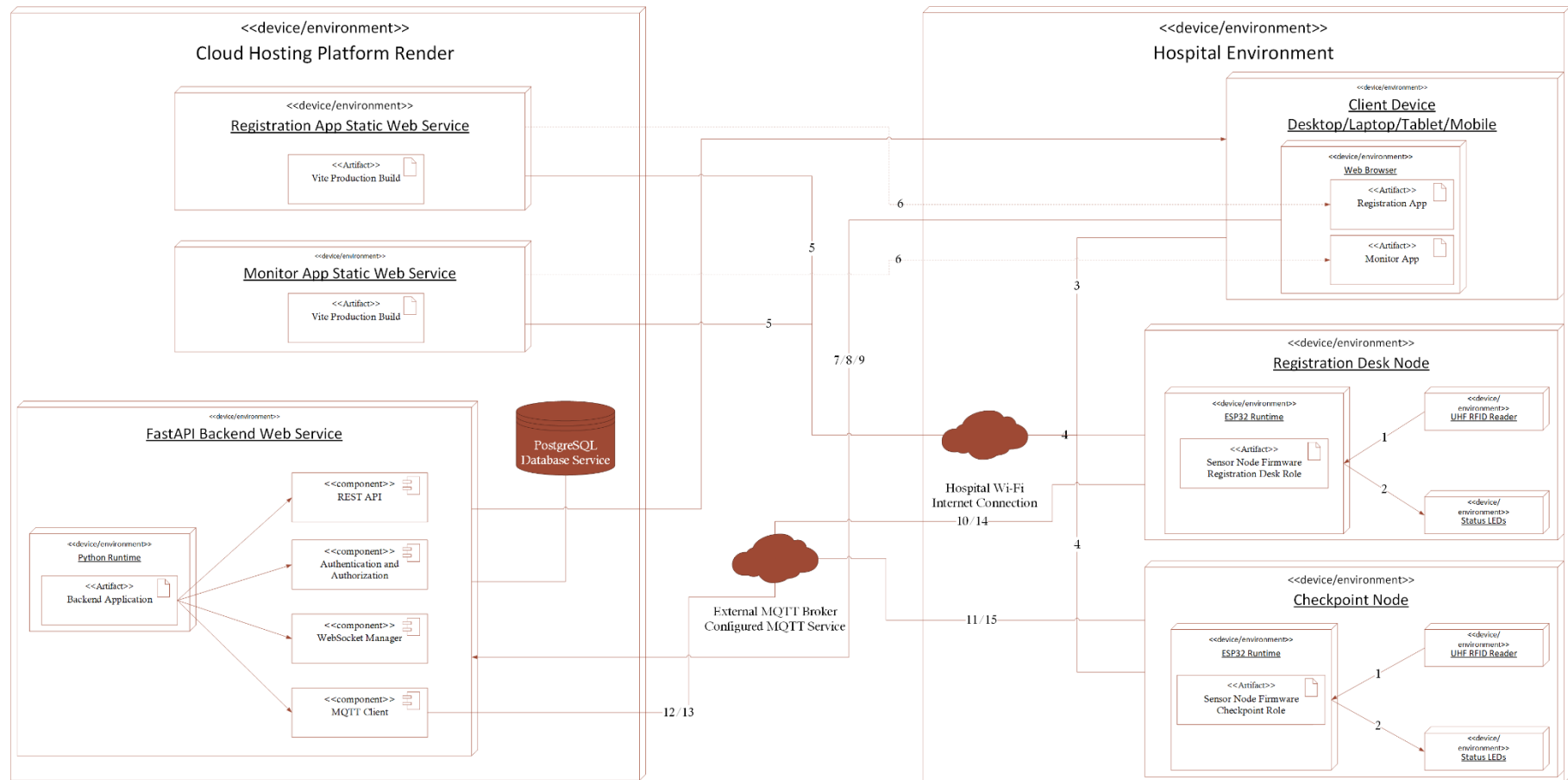


Figure 11. 1 Complete System Deployment Diagram.

The numeric relation lines between each device and environment in the diagram above can be elaborated as follows.

Table 11. 2 List of relations and communication lines of the system deployment diagram.

No.	Relation	Data Communication
1.	Registration Desk UHF RFID Reader to Registration Desk ESP32 Runtime	UART/Serial
	Checkpoint Node UHF RFID Reader to Checkpoint Node ESP32 Runtime	
2.	Registration Desk ESP32 Runtime to Registration Desk Status LED	GPIO
	Checkpoint Node ESP32 Runtime to Checkpoint Node Status LED	
3.	Client Device to Hospital Wi-Fi/Internet Connection	Wi-Fi/LAN
4.	Registration Desk Node to Hospital Wi-Fi/Internet Connection	Wi-Fi
	Checkpoint Node to Hospital Wi-Fi/Internet Connection	
5.	Hospital Wi-Fi/Internet Connection to Registration App Static Web Service	HTTPS Load static application
	Hospital Wi-Fi/Internet Connection to Monitor App Static Web Service	
6.	Registration App Static Web Service to Registration App	Deliver application files
	Monitor App Static Web Service to Monitor App	
7.	Web Browser to FastAPI Backend Web Service	HTTPS/JSON REST API requests
8.	FastAPI Backend Web Service to Web Browser	HTTPS/JSON responses and errors
9.	FastAPI Backend Web Service to Web Browser	WSS/WebSocket live system updates
10.	Registration Desk Node to External MQTT Broker Configured MQTT Service	MQTT registration scans, heartbeats, and ACKs
11.	Checkpoint Node to External MQTT Broker, Configured MQTT Service	MQTT RFID detections, heartbeats, and ACKs
12.	External MQTT Broker Configured MQTT Service to MQTT Client	MQTT delivers node messages
13.	MQTT Client to External MQTT Broker Configured MQTT Service	MQTT publishes node commands
14.	External MQTT Broker Configured MQTT Service to Registration Desk Node	MQTT blink or supported commands
15.	External MQTT Broker Configured MQTT Service to Checkpoint Node	MQTT blink or supported commands

And the simplified version of the overall deployment diagram is shown below.

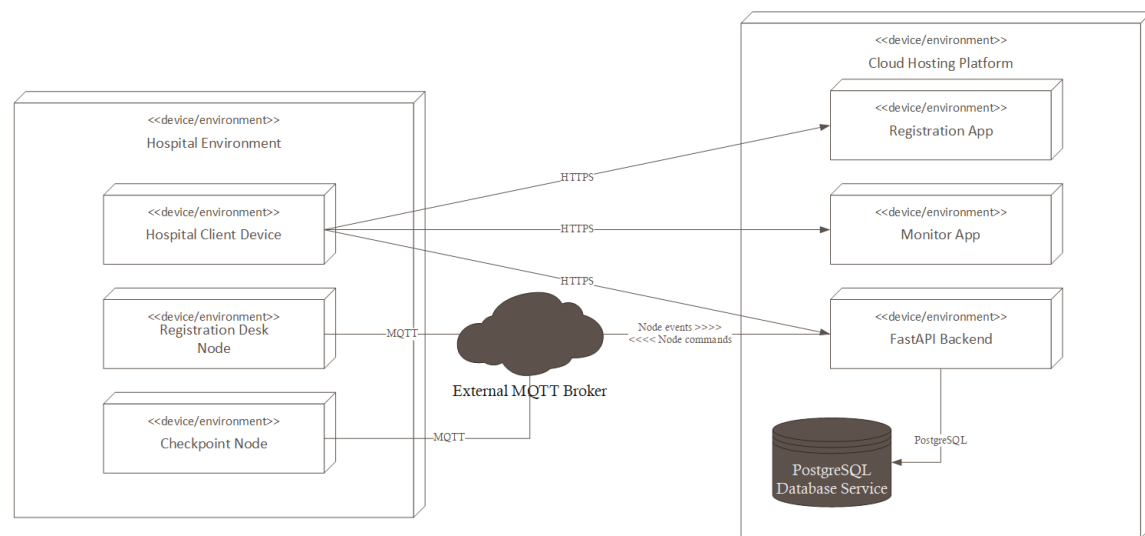


Figure 11. 2 Simplified Complete System Deployment Diagram.

The complete deployment diagrams are in the System Design Document.

12. Logging, Audit, and Operational Monitoring

Logging and operational monitoring requirements define which technical failures and business events shall be recorded, which sensitive values must be excluded, how timestamps are handled, and what health information maintainers need in production. User-facing activity records and technical logs serve different audiences and may use different retention policies.

Table 12. 1 System data log requirements.

No.	ID	Logging/Monitoring Requirement
1.	LOG-001	The backend shall log startup, shutdown, and configuration failures.
2.	LOG-002	The backend shall log MQTT connection, subscription, parsing, and publication failures.
3.	LOG-003	The backend shall log database connection and transaction failures without exposing credentials.
4.	LOG-004	The system shall record significant user management actions, including account creation, role changes, activation, deactivation, and password resets, where applicable.
5.	LOG-005	The system shall record movement requests, cancellations, approvals, rejections, and completions.
6.	LOG-006	The system shall record significant changes in node status and assignments.
7.	LOG-007	The system shall record asset registration, deregistration, valid placement, and warning conditions.
8.	LOG-008	Logs shall not include plain-text passwords, bearer tokens, database passwords, or provisioning secrets.
9.	LOG-009	Log timestamps shall use a consistent time basis.
10.	LOG-010	User-facing activity records shall remain understandable without exposing unnecessary technical stack traces.
11.	LOG-011	Production deployment should provide a method to inspect service health, recent errors, and connection status.
12.	LOG-012	Log retention and access permissions shall be defined by deployment policy before production acceptance.

13. Verification & Acceptance Criteria

Verification shall combine inspection, analysis, demonstration, automated tests, integration tests, and end-to-end acceptance tests. A mandatory requirement is accepted only when its expected result is demonstrated in the defined test environment and recorded.

Performance, availability, scalability, retention, and recovery targets marked once a month shall be agreed upon and added before formal production acceptance.

Table 13. 1 List of system verification and acceptance criteria.

No.	ID	Test Scenario	Related Requirement(s)	Expected Result
1.	AC-001	Start with a database containing no admin and submit valid first-admin details.	• FR-069	One active admin is created, and the setup flow becomes unavailable.
2.	AC-002	Test valid login, invalid password, disabled account, role restrictions, logout, and password change.	• FR-001 • FR-002 • FR-003 • FR-004 • FR-005 • FR-006 • FR-007 • FR-008 • FR-009	Authentication and authorization results match the specified rules.
3.	AC-003	Discover, assign, edit, identify, unassign, and delete eligible nodes.	• FR-014 • FR-015 • FR-016 • FR-017 • FR-018 • FR-019 • FR-020 • FR-021	Each action updates the correct node state, and invalid actions are rejected.
4.	AC-004	Send a blink command to an online node.	• FR-019	The node blinks the configured LEDs and either receives an acknowledgment or a clear failure status.

5.	AC-005	Scan a tag, register an asset, attempt to register it again, then deregister it.	• FR-024 • FR-025 • FR-026 • FR-027 • FR-028 • FR-029 • FR-030	The asset lifecycle follows the requirements, and duplicates are blocked.
6.	AC-006	Create, cancel, approve, and reject movement requests in valid and invalid states.	• FR-031 • FR-032 • FR-033 • FR-034 • FR-035 • FR-036 • FR-037 • FR-038 • FR-039	Only permitted transitions succeed, and both applications display the result.
7.	AC-007	Detect an in-transit asset at its approved destination.	• FR-040 • FR-041	The movement is completed, and the assigned/current room plus flow status are updated.
8.	AC-008	Detect an asset outside the expected location without an approved workflow.	• FR-050 • FR-051	An unauthorized movement or wrong-location warning is displayed.
9.	AC-009	Stop heartbeat publication beyond the configured timeout.	• FR-048 • FR-049 • FR-054	The node goes offline, and an alert appears.
10.	AC-010	Present an unknown tag to a valid checkpoint node.	• FR-047	The event is handled safely, and an unknown-tag alert is displayed.
11.	AC-011	Interrupt and restore WebSocket connectivity.	• FR-057 • FR-058 • FR-059 • FR-060 • FR-061	The client shows connection state, reconnects, and resumes supported live updates.
12.	AC-012	Load and update valid and invalid settings using authorized and unauthorized roles.	• FR-064 • FR-065 • FR-066 • FR-067 • FR-068	Only authorized, valid changes are stored.
13.	AC-013	Test representative pages at desktop, half-window, tablet, and mobile widths.	• NFR-023	Navigation, forms, tables, and dialogs remain usable without critical controls being clipped.

14.	AC-014	Inject failures for API, database, MQTT, WebSocket, validation, and provisioning.	• ERR-001 • ERR-002 • ERR-003 • ERR-004 • ERR-005 • ERR-006 • ERR-007 • ERR-008 • ERR-009 • ERR-010 • ERR-011 • ERR-012 • ERR-013 • ERR-014 • ERR-015	Failures are reported clearly and do not cause an inconsistent persistent state.
15.	AC-015	Deploy a clean development or production-like environment using the documented checklist.	• DEP-001 • DEP-002 • DEP-003 • DEP-004 • DEP-005 • DEP-006 • DEP-007 • DEP-008 • DEP-009 • DEP-010 • DEP-011 • DEP-012 • DEP-013 • DEP-014 • DEP-015	The system starts with all required services, and no secrets are committed to source control.

14. Requirements Traceability Matrix

This matrix connects the user requirements maintained in the Use Case Specification to system functional requirements and representative acceptance tests. The complete design-to-test traceability version may be maintained in the project Test Repository and the Acceptance Report document.

Table 14. 1 Matrix table of user requirements with use case and system acceptance evidence.

No.	User Requirement(s)	Functional Requirement(s)	Acceptance Evidence
1.	UR-001 – Authentication and role access	- FR-001 - FR-002 - FR-003 - FR-004 - FR-009 - FR-062 - FR-063	AC-002
2.	UR-002 – Change own password	- FR-005 - FR-006	AC-002
3.	UR-003 – Manage user accounts	- FR-007 - FR-008	AC-002
4.	UR-004 – Role-based registration dashboard	- FR-010 - FR-011 - FR-012	- AC-002 - AC-013
5.	UR-005 – Access node registration	- FR-013 - FR-062 - FR-063	AC-003
6.	UR-006 – View node records	- FR-014 - FR-015 - FR-044	AC-003
7.	UR-007 – Manage and identify nodes	- FR-016 - FR-017 - FR-018 - FR-019 - FR-020 - FR-021	- AC-003 - AC-004
8.	UR-008 – Access asset registration	- FR-022 - FR-023	AC-005
9.	UR-009 – Register RFID-tagged assets	- FR-024 - FR-025 - FR-026 - FR-027 - FR-028 - FR-052	AC-005
10.	UR-010 – Deregister assets	- FR-029 - FR-030	AC-005
11.	UR-011 – Request and cancel movement	- FR-031 - FR-032 - FR-033 - FR-034	AC-006
12.	UR-012 – Approve or reject movement	- FR-035 - FR-036 - FR-037 - FR-038 - FR-039	AC-006
13.	UR-013 – Complete approved movement	- FR-040 - FR-041 - FR-053	AC-007

14.	UR-014 – Monitor assets, nodes, alerts, and activities	• FR-042 • FR-043 • FR-044 • FR-045 • FR-046	• AC-009 • AC-013
15.	UR-015 – Detect and show warnings	• FR-047 • FR-048 • FR-049 • FR-050 • FR-051	• AC-008 • AC-009 • AC-010
16.	UR-016 – Receive node events through MQTT	• FR-052 • FR-053 • FR-054 • FR-055 • FR-056	• AC-004 • AC-005 • AC-007 • AC-008 • AC-009 • AC-010
17.	UR-017 – Broadcast live updates	• FR-057 • FR-058 • FR-059 • FR-060 • FR-061	• AC-011
18.	UR-018 – Prevent unauthorized access	• FR-009 • FR-062 • FR-063 • FR-068	• AC-002 • AC-012
19.	UR-019 – Manage system settings	• FR-064 • FR-065 • FR-066 • FR-067 • FR-068	• AC-012
20.	UR-020 – Create first admin	• FR-069	• AC-001

Appendices

These appendices provide controlled reference information to support implementation and review without altering the normative requirement statements above.

Appendix A – Status and Role Enumerations

This appendix defines the expected role and status values used across backend constants, frontend badges, tests, diagrams, and documentation. Stored values shall remain consistent with the implementation.

Table 15. 1 System status and role rules.

No.	Category	Allowed/Expected Values
1.	User Roles	admin, test_user, technician, registration_staff, monitor_staff
2.	Node Roles	Registration Desk, Checkpoint Node
3.	Node Statuses	DISCOVERED, ASSIGNED, ONLINE, OFFLINE, DISABLED, UNKNOWN
4.	Movement Request Statuses	PENDING, APPROVED, REJECTED, CANCELLED, COMPLETED
5.	Representative Asset Flow States	Pending Placement, In Place, Pending Approval, In Transit, Unauthorized Movement, Rejected

Appendix B – Recommended Figures

This appendix defines the expected role and status values used across backend constants, frontend badges, tests, diagrams, and documentation. Stored values shall remain consistent with the implementation.

Table 15. 2 Document figures.

No.	Figure	Recommended Location	Purpose
1.	Figure 1 – System Context Diagram	Section 2.1	Shows the product boundary, human actors, nodes, backend, database, and broker.
2.	Figure 2 – High-Level Software Architecture	Section 3	Shows the product boundary, human actors, nodes, backend, database, and broker.
3.	Figure 3 – Sensor Node Hardware Wiring Diagram	Section 6.1	Shows ESP32, RFID reader, power, antenna, LEDs, and controls.
4.	Figure 4 – Communication Flow Diagram	Section 6.3	Shows MQTT, HTTPS/API, and WebSocket directions and boundaries.
5.	Figure 5 – Entity-Relationship Diagram	Section 7.1	Shows persistent entities and relationships.
6.	Figure 6 – Deployment Diagram	Section 11	Shows hosted services, hospital network, browsers, and sensor nodes.

Appendix C – Open Decisions and TBD Values

The following values require stakeholder or deployment approval before production acceptance. Until approved, they remain explicitly open decisions rather than implied guarantees.

Table 15. 3 List of decision plans for the product.

No.	Item	Decision Required
1.	Performance target	Maximum acceptable page/API response time and live-update delay.
2.	Availability target	Required production uptime and operating hours.
3.	Scalability target	Expected number of active nodes, assets, users, and event rate.
4.	Retention policy	Retention periods for detections, alerts, activity, and logs.
5.	Backup and recovery	Backup frequency, recovery point objective, and recovery time objective.
6.	MQTT security	Production broker authentication, TLS requirement, topic authorization, and credential rotation.
7.	Compliance assessment	Hospital IT, privacy, electrical, radio, and regulatory review are required for deployment.